

Online Ride Dispatch Optimizer

Technical Report

Lola Rapoport

May 21, 2026

Abstract

This report describes a simulator for real-time ride dispatch on a graph. Riders arrive online, drivers move through a road-network abstraction, and dispatch policies must assign available drivers to pending riders while balancing pickup cost, rider waiting time, rejection rate, and fleet utilization. The project implements five policies: greedy nearest-driver matching, fixed-interval batch matching, min-cost bipartite matching, an online primal-dual style heuristic, and a learned demand-aware heuristic. The simulator is intended as a practical demonstration of online algorithms, matching, stochastic demand modeling, and operational tradeoffs relevant to algorithmic mobility platforms.

1 Motivation

Ride dispatch is an online decision problem: requests arrive without full knowledge of the future, and each assignment consumes a scarce resource, namely driver availability in space and time. A locally cheap assignment can create future scarcity in a high-demand zone, while excessive batching can improve optimization quality at the cost of rider delay. This makes the domain a good applied setting for online algorithms, primal-dual thinking, min-cost assignment, and empirical evaluation.

2 System Model

The city is represented as an undirected graph

$$G = (V, E),$$

where each node is a location and each edge has unit travel time. The implementation uses a grid graph by default, but the simulator is written against NetworkX graphs and can support richer road-network inputs.

At each discrete time step $t = 0, 1, \dots, H - 1$, a random number of riders arrives. A rider

$$r = (o_r, d_r, a_r, w_r^{\max})$$

has origin $o_r \in V$, destination $d_r \in V$, arrival time a_r , and maximum allowed wait $w_r^{\max}$. A driver

$$i = (x_i(t), b_i(t))$$

has current node $x_i(t)$ and busy-until time $b_i(t)$. Driver i is available at time t when $b_i(t) \leq t$.

The shortest-path distance between two graph nodes is denoted by

$$\text{dist}(u, v).$$

If driver i is assigned to rider r at time t , the pickup time is

$$p_{ir}(t) = \text{dist}(x_i(t), o_r),$$

the trip time is

$$\tau_r = \text{dist}(o_r, d_r),$$

and the rider's total wait is modeled as

$$W_r(t) = (t - a_r) + p_{ir}(t).$$

The driver becomes unavailable until $t + p_{ir}(t) + \tau_r$, then reappears at d_r .

3 Objective

The simulator evaluates policies empirically instead of solving a single offline objective. The central tradeoff is captured by the following stylized cost:

$$\min \sum_{r \in M} (\alpha W_r + \beta p_{ir}) + \lambda \sum_{r \in R} \mathbf{1}\{r \text{ rejected}\} - \gamma \cdot \text{Utilization},$$

where M is the set of matched riders and R is the set of all arrived riders. In practice, the report focuses on interpretable metrics:

- mean rider wait time,
- 95th percentile rider wait time,
- mean pickup distance,
- rejection rate,
- completion rate,
- fleet utilization.

4 Arrival Process

Rider arrivals are generated by a non-homogeneous Poisson process. The expected arrivals at time t are

$$\lambda(t) = \lambda_0 (1 + m \cdot q(t)),$$

where λ_0 is the base arrival rate, m is a peak multiplier, and $q(t)$ is a periodic demand curve. Spatial demand is biased toward configurable hot spots. This creates more realistic imbalances than uniform arrivals and allows the demand-aware policy to have a meaningful signal.

5 Algorithms

5.1 Greedy Nearest-Driver Matching

The greedy policy processes pending riders in arrival order and assigns each rider the closest currently available driver:

$$i^*(r) = \arg \min_{i \in A(t)} \text{dist}(x_i(t), o_r).$$

It is fast, online, and easy to reason about, but it does not account for the joint structure of multiple simultaneous riders. It can make locally optimal choices that are globally suboptimal.

5.2 Batch Matching Every T Seconds

The batch policy only matches at times divisible by a fixed interval T . At batch time, it solves a min-cost assignment over the currently available drivers and pending riders. Batching can improve matching quality when several riders are present simultaneously, but it intentionally delays riders between batch boundaries.

5.3 Min-Cost Bipartite Matching

At every time step, the min-cost policy builds a bipartite graph between available drivers and pending riders. The assignment cost is

$$c_{ir}(t) = \alpha \text{dist}(x_i(t), o_r) - \eta(t - a_r),$$

where the wait credit η prioritizes riders who have already waited. The implementation solves the linear assignment problem with SciPy’s Hungarian algorithm:

$$\min \sum_{i,r} c_{ir}(t) x_{ir}$$

subject to each driver and rider being used at most once.

5.4 Online Primal-Dual Style Heuristic

The primal-dual inspired policy maintains nonnegative zone-level shadow prices. Each zone z receives a dual variable $\pi_z(t)$ that increases when pending rider demand exceeds available driver supply:

$$\pi_z(t+1) = \max\{0, \rho\pi_z(t) + s(D_z(t) - S_z(t))\},$$

where $D_z(t)$ is pending demand, $S_z(t)$ is available supply, ρ is a decay factor, and s is a step size.

The assignment score adjusts pickup cost using both the driver’s current zone and the rider’s origin zone:

$$\text{score}_{ir}(t) = \text{dist}(x_i(t), o_r) + \pi_{z(x_i)}(t) - 1.25\pi_{z(o_r)}(t) - \eta(t - a_r).$$

This heuristic discourages draining drivers from scarce zones and encourages serving high-pressure zones.

5.5 Learned Demand-Aware Heuristic

The learned demand-aware policy uses an online exponentially decayed demand forecast. For each zone z ,

$$\hat{d}_z(t+1) = \delta\hat{d}_z(t) + A_z(t),$$

where $A_z(t)$ is the number of new arrivals in zone z and δ is a decay rate. The assignment cost reserves drivers in high-demand zones and rewards trips that end near predicted future demand:

$$c_{ir}(t) = \text{dist}(x_i(t), o_r) + \mu\hat{d}_{z(x_i)}(t) - \nu\hat{d}_{z(o_r)}(t) - \eta(t - a_r).$$

This is intentionally lightweight rather than a full supervised model; it demonstrates the architecture needed to replace the forecast with a trained demand model later.

6 Complexity

Let m be the number of available drivers and n be the number of pending riders at a time step.

- Greedy nearest-driver matching computes pairwise distances in $O(mn)$ time per step.
- Min-cost bipartite matching builds an $m \times n$ cost matrix and solves assignment in $O(k^3)$, where $k = \max(m, n)$, after distance lookup.
- Batch matching has the same per-batch complexity as min-cost matching but runs only every T steps.
- The primal-dual heuristic updates zone prices in $O(m + n)$, then sorts candidate pairs, giving $O(mn \log(mn))$ in the current implementation.
- The demand-aware heuristic has the same assignment complexity as min-cost bipartite matching plus lightweight forecast updates.

Shortest-path distances are cached by an all-pairs distance oracle for the grid graph, trading memory for fast repeated dispatch queries.

7 Experimental Setup

The default experiment uses:

- a 16×16 grid graph,
- 45 drivers,
- 600 time steps,
- base rider arrival rate 0.28,
- maximum rider wait of 120 seconds,
- batch interval $T = 30$ seconds,
- fixed random seed for reproducibility.

The experiment can be reproduced with:

```
python -m ride_dispatch_optimizer.cli \  
  --horizon 600 \  
  --drivers 45 \  
  --grid-width 16 \  
  --grid-height 16 \  
  --base-rate 0.28 \  
  --out outputs/latest
```

8 Demonstrated Results

The repository includes a demonstrated result snapshot under `docs/`. A fresh experiment writes updated artifacts under `outputs/latest/`; those can be copied into `docs/` when the report snapshot should be refreshed.

9 Interpretation

The min-cost bipartite policy generally provides the strongest immediate matching quality because it optimizes assignments jointly instead of greedily. Greedy matching is competitive when demand is light and the fleet is well distributed, but it lacks a mechanism for resolving conflicts among nearby riders. Batch matching exposes the central latency-quality tradeoff: delaying decisions

Table 1: Policy comparison from the latest generated experiment.

policy	completed	rejected	mean_wait_time	p95_wait_time	mean_pickup_distance	fleet_utilization
min-cost-bipartite	307	0	2.059	6.000	2.059	0.143
online-primal-dual	307	0	2.075	6.000	2.075	0.143
greedy-nearest	307	0	2.104	6.000	2.104	0.144
learned-demand-aware	307	0	2.121	5.000	2.121	0.144
batch-30s	298	9	16.091	30.000	2.107	0.139

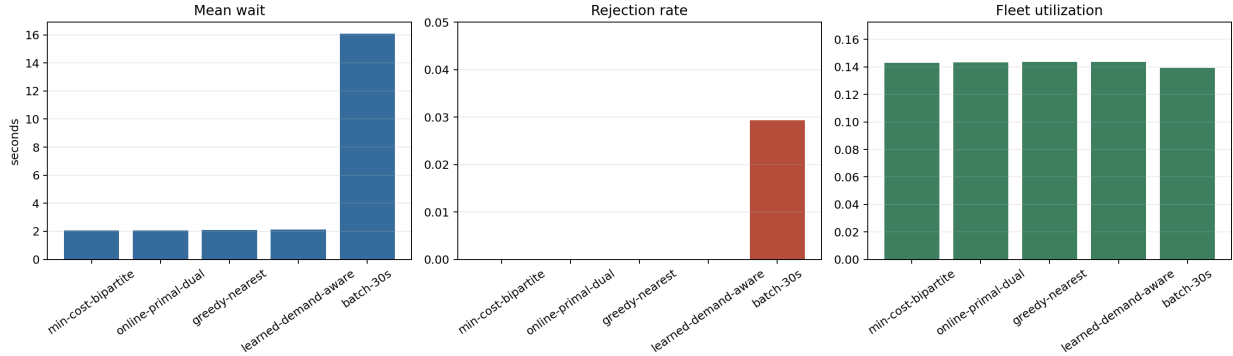


Figure 1: Comparison of mean wait time, rejection rate, and fleet utilization across dispatch policies.

can create better assignment opportunities, but it can also increase rider wait and rejection risk. The primal-dual and demand-aware policies are most interesting under spatial imbalance, where preserving supply in high-value zones can matter more than minimizing the current pickup distance alone.

10 Limitations

The simulator intentionally abstracts away several production details:

- travel times are graph distances rather than traffic-dependent estimates,
- drivers do not strategically reposition except for random idle movement,
- demand learning is an online heuristic rather than a trained model,
- cancellations, pooling, driver preferences, and pricing are not modeled,
- the grid graph is a simplified road network.

These abstractions keep the project focused on online matching and dispatch policy evaluation while leaving room for realistic extensions.

11 Future Work

Natural extensions include adversarial arrival sequences, competitive-ratio experiments, driver repositioning, fairness constraints across regions, larger road networks, sensitivity analysis over fleet size and batch interval, and learned demand models trained from historical traces.

12 Conclusion

The project demonstrates how online algorithms and matching theory can be translated into a practical simulator for real-time ride dispatch. It provides a clean experimental framework for comparing local greedy decisions, batch optimization, min-cost assignment, primal-dual-style pressure balancing, and demand-aware forecasting under stochastic arrivals.